

김혜진 ✦

Frontend Developer.

사용자도, 팀도 막히지 않게 만드는 프론트엔드 개발자입니다 👨‍💻

사용자가 막히는 곳을 출시 전에 찾고, 실제 사용자 앞에서 문제를 해결해 왔습니다. 디엔소프트에서 **3일간 754명**이 참여한 박람회 이벤트의 현장 장애 대응을 맡았고, **100개 이상의 초등학교**가 사용한 관리자 시스템을 일정 안에 오픈했습니다.

측정으로 병목을 찾고, 팀이 기다리지 않게 준비합니다 💪

다른 파트가 늦어져도 팀이 기다리지 않도록 **공통 스타일과 MSW 목 API**를 먼저 준비하고, 성능은 측정으로 병목을 찾아 고칩니다. NEXT GACHA 홈을 서버 렌더링으로 바꿔 **모바일 실측 LCP를 3.28s에서 1.78s**로 줄였습니다.

About

✉ khj000217@gmail.com

⌘ https://github.com/llmeajinll

✎ https://velog.io/@llmeajinll

Skills

- ✦ React, TypeScript, Next.js, Vue.js
- ✦ TanStack Query, Zustand, Jotai, Vanilla Extract, Tailwind CSS
- ✦ MSW, Storybook, Vitest, Jest, Playwright 외 여러 도구

Nova

2026.09.15 ~ 2026.10.28

Site ↗ GitHub ↗



개요 📖

스마트폰 사전예약 플랫폼 · LG유플러스 유레카 최종융합프로젝트. 오픈 직후 신청이 몰려도 사전예약을 유실 없이 접수하고, 외부 시스템 처리를 기다리지 않고 결과를 안내하는 스마트폰 사전예약 플랫폼

- **역할** Frontend (구매자 화면 전담)
- **인원** 7명 (백엔드 5, 프론트엔드 2)
- **기여도** 60% (프론트엔드 커밋 기준)

역할 👨‍💻

- **구매자 화면** 메인, 사전예약 목록·상세, 상품 상세·옵션 선택, 사전예약 대기열, 결제 (토스페이먼츠), 주문 결과, 마이페이지(장바구니·배송지·사전예약 내역)
- **구매자 화면** 카카오 로그인·회원가입 온보딩, 세션 관리
- **프론트엔드 기반** FSD(Feature-Sliced Design) 레이어 구조 설계, ESLint로 레이어 의존 방향 강제
- **프론트엔드 기반** API 클라이언트(타임아웃·요청 취소·에러 변환), TanStack Query 캐시·재시도 정책
- **프론트엔드 기반** MSW 목 서버와 실패 재현 도구, 디자인 토큰·공용 UI 컴포넌트

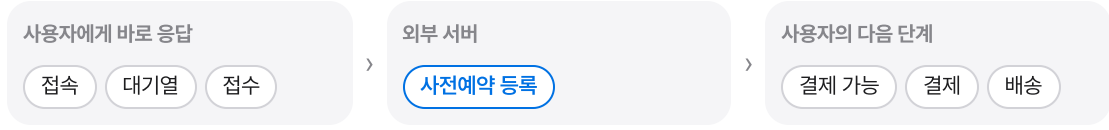
Skills 🛠️

- React 19, TypeScript, Vite, TanStack Query v5, Zustand, Axios
- Vanilla Extract, react-router v7, MSW 2, 토스페이먼츠 SDK, 카카오 OAuth, Storybook 10 + Vitest
- Playwright, Amplitude, ESLint, husky + commitlint, Vercel

기획 배경 📋

이 프로젝트의 조건은 까다로웠습니다. 스마트폰 사전예약이 열리면 **10초 안에 5,000건**이 몰립니다. 예약을 확정하려면 외부 예약 시스템에 등록해야 하는데, 이 시스템은 **응답에 평균 0.5초가 걸리고 5%는 실패**합니다. 그런데 요구사항은 **신청 유실 0건, 중복 예약 0건, 외부 시스템과 우리 서버의 기록이 100% 일치**하는 것이었습니다.

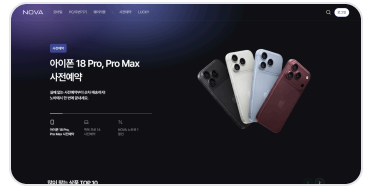
외부 시스템의 응답을 기다렸다가 답하면, 5,000명이 모두 0.5초씩 기다려야 하고 실패하는 5%는 신청 자체가 실패합니다. 그래서 일을 둘로 나눴습니다. **접수**는 "신청을 받았습니다. 3,215번입니다"처럼 순번만 바로 알려 주는 일이고, **확정**은 그 뒤에 서버가 외부 예약 시스템에 등록해서 진짜 예약으로 만드는 일입니다. 인기 맛집에서 번호표부터 뽑아 주고 자리 배정은 나중에 하는 것과 같습니다. 외부 시스템이 느리거나 멈춰도 접수는 막히지 않습니다.



접수까지는 사용자에게 바로 응답하고, 외부 서버에 등록하는 일(확정)은 뒤에서 처리합니다. 결제는 등록이 끝난 뒤에 열립니다.

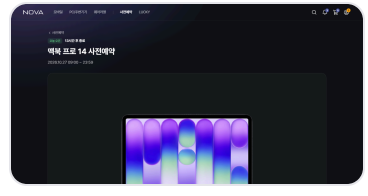
외부 등록이 실패해도 신청은 사라지지 않습니다. 일시적인 실패는 서버가 자동으로 다시 시도하고, 그래도 안 되면 운영자가 확인해서 다시 보냅니다. 같은 신청이 두 번 들어와도 예약은 하나만 만들어지고, 서버가 매일 새벽 두 기록을 대조해 어긋난 건을 운영자가 바로 고칠 수 있습니다. 프론트엔드는 이 두 단계를 화면에서 **다른 상태로 보여주는 것**을 맡았습니다.

주요 기능 플로우 🧭



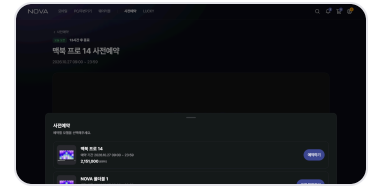
01 메인

진행 중인 사전예약을 배너로 보여줍니다.



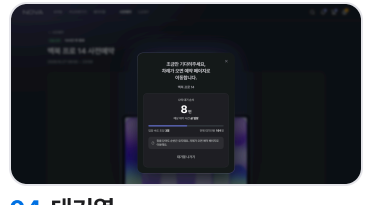
02 사전예약 상세

일정과 모델을 확인하고 [사전예약 하러가기]로 시작합니다.



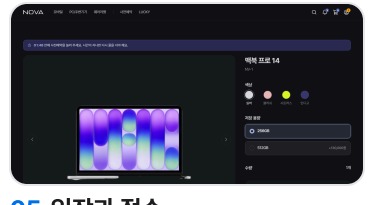
03 모델 선택

모델을 고릅니다. 이미 예약한 모델은 [구매 확정]으로 바뀝니다.



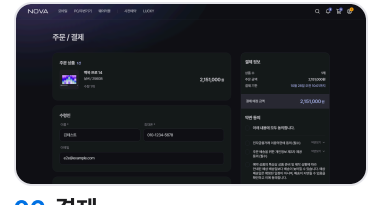
04 대기열

내 순서와 예상 대기 시간을 보여줍니다. 창을 닫아도 순번은 유지되고, 차례가 오면 예약 페이지로 이동합니다.



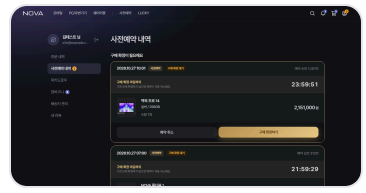
05 입장과 접수

입장권 유효 시간 안에 옵션을 골라 접수합니다. 남은 시간은 상단에 계속 보입니다.



06 결제

결제 기한을 안내하고 토스 결제창을 엽니다.



07 사전예약 내역

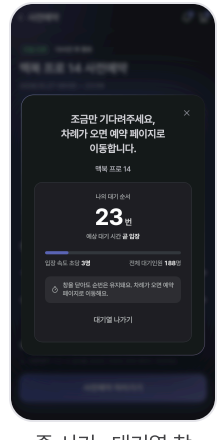
예약 순번과 구매 확정 마감 카운트다운을 보여줍니다.

접수 뒤에는 외부 서버 등록을 최대 5초 기다립니다

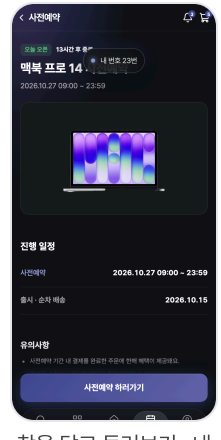
- ✦ **5초 안에 끝나면** 5초 안에 등록이 끝나면 바로 결제 페이지로 이동합니다.
- ✦ **5초를 넘기면** 5초를 넘기면 예약 접수 확인 화면으로 이동해, 지금 처리 단계와 처리 기록을 보여줍니다.

모바일 화면 📱

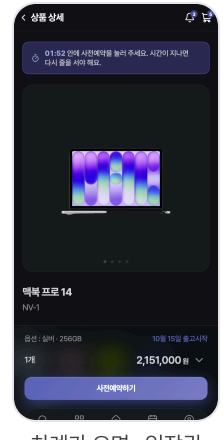
대기열 창을 닫아도 순번은 유지되고, 내 번호가 담긴 pill이 상단에 남아 다른 페이지를 둘러볼 수 있습니다.



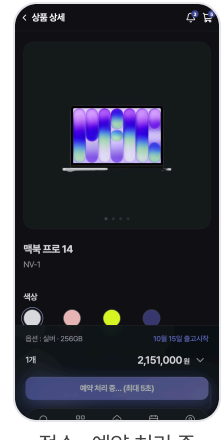
줄 서기 · 대기열 창



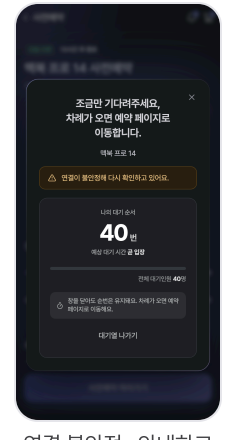
창을 닫고 둘러보기 · 내 번호 pill



차례가 오면 · 입장권 타이머



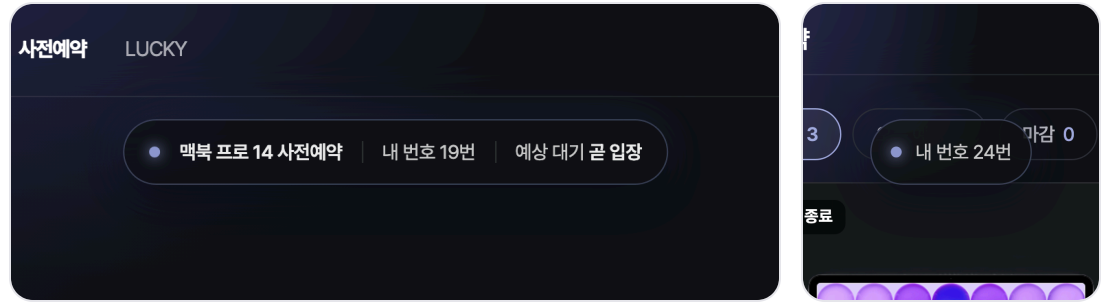
접수 · 예약 처리 중



연결 불안정 · 안내하고 다시 확인

핵심 구현 💡

사전예약 대기열: 창을 닫아도 따라오는 순번 pill과 상태별 안내



대기열 창을 닫고 다른 페이지로 이동해도 상단에 pill이 따라옵니다.

구현 조건

- ✦ 대기열 창을 꺼도 상단 pill에 내 대기 순번이 보여야 합니다.
- ✦ pill이 떠 있는 상태로 다른 페이지로 이동하며 다른 상품들을 구경할 수 있어야 합니다.
- ✦ 대기열 창에서는 현재 서버 상태에 따라 보여지는 안내 문구가 달라야 합니다.

구현

대기열 창을 페이지 안이 아니라 **앱 맨 위(`RootLayout`)에 한 곳만** 두었습니다. 줄을 서는 쪽은 `joinPreorderQueue()` 만 부르면 되는, 토스트와 같은 구조입니다. 그래서 창을 닫아도 줄은 유지되고, 다른 페이지로 가도 내 번호가 담긴 작은 pill(`QueuePill`)이 화면에 따라옵니다.
새로고침하거나 창을 닫아도 같은 자리, 같은 키로 이어집니다.

- ✦ **pill 문구**: 확인 전 "내 번호 확인 중", 대기 중 "내 번호 23번", 거의 다 되면 "곧 내 차례예요 · 내 번호 23번", 차례가 오면 "내 차례예요 · 곧 예약 페이지로 이동해요"와 남은 초.
- ✦ **차례가 와도 화면이 갑자기 바뀌지 않게 했습니다.** 사용자가 맥락을 놓치지 않도록 시간적 여유를 두었습니다. 창이 열려 있을 때는 진행률 100%를 0.8초간 보여준 뒤 이동하고, 창이 닫혀 있을 때는 상단 pill에서 5초를 센 뒤 자연스럽게 전환됩니다.

대기열 창 상태별 안내 문구

서버 상태	대기열 창의 안내
순번이 줄어드는 중 `status: "WAITING"` · `position: 23` · `totalWaiting` 헤더 `Retry-After: 3`	"조금만 기다려주세요, 차례가 오면 예약 페이지로 이동합니다."
내 차례가 옴	"내 차례예요, 곧 예약 페이지로 이동합니다."
순번 조회가 잠깐 실패	"연결이 불안정해 다시 확인하고 있어요." (순번은 마지막 값 유지)
인터넷이 끊김	"인터넷 연결이 끊겼어요. 연결되면 자동으로 이어서 확인해요."
오래 끊겨 줄에서 빠졌을 수 있음	"연결이 오래 끊겨 줄에서 빠졌을 수 있어요. 연결되면 바로 다시 확인할게요."
끊긴 사이 줄에서 빠져 다시 섬	"연결이 끊긴 사이 줄에서 빠져 다시 줄을 섰어요 (40번 → 52번)."
새로고침 뒤 같은 자리로 돌아옴	"전에 서 있던 자리 그대로예요. 순번을 잃지 않았어요."

- **줄에 들어가지 못하면 다시 시도합니다.** 줄이 가득 차면(`QUEUE_FULL`) 10초 뒤, 일시 장애면 2초 뒤 다시 진입합니다.
- **예상 대기 시간은 초가 아니라 구간으로 보여줍니다.** "곧 입장 / 약 1분 / 약 5분 / 오래 걸림"입니다. "0초"가 보이면 사용자가 곧 입장하는 것으로 오해하기 때문입니다.
- **타이머가 리셋되지 않게 했습니다.** 호출하는 쪽이 렌더링마다 새 함수를 넘겨도 카운트가 처음부터 다시 시작되지 않도록 React 19의 `useEffectEvent` 를 썼습니다.

"접수됐다"와 "확정됐다"를 화면에서 다르게 보여주기

구현 조건

- 접수(순번 확보)와 확정(외부 등록 완료)을 화면에서 다른 상태로 보여줘야 합니다.
- 외부 등록이 끝나기 전에는 결제를 열지 않아야 합니다.
- 등록이 늦어져도 사용자가 아무 안내 없이 기다리게 해서는 안 됩니다.

구현

서버의 예약 단계(`displayStatus`)를 그대로 따라 문구와 버튼을 정했습니다. 결제가 필요한 단계만 노란색으로 눈에 띄게 하고, 그 전 단계에서는 결제를 열지 않습니다.

단계	화면에서 보이는 말	의미
`RECEIVED` · `PROCESSING`	접수 완료 · 예약 처리 중	접수만 끝났고 아직 결제할 수 없음. "잠시 뒤 구매 확정 가능"
`PAYABLE`	구매 확정 대기	외부 등록이 끝나 결제 가능. 24시간 카운트다운
`PAYMENT_IN_PROGRESS`	결제 진행 중	결제창을 열었다 닫아도 다시 결제해야 함
`PAYMENT_EXPIRED`	결제 기한 지남	기한이 지나 취소 처리를 시작함
`RESERVED`	예약 확정	결제까지 끝남
`CANCELING` · `CANCELED`	취소 처리 중 · 취소 완료	결제한 예약이면 환불이 끝나야 취소 완료

- ✦ **접수 직후 5초만 기다려 길을 가릅니다.** 1초마다 외부 등록이 끝났는지 묻고, 5초 안에 끝나면 바로 결제로 보냅니다. 늦어지면 예약 접수 화면으로 보내 이어서 기다립니다. 5초가 되면 진행 중인 조회도 끊어서, 늦게 도착한 응답이 사용자를 결제로 끌고 가지 않게 했습니다.
- ✦ **등록이 끝내 안 끝나는 경우를 안내합니다.** 처리 중에는 3초마다 다시 묻고, 접수 후 60초가 지나면 "등록이 평소보다 오래 걸리고 있어요. 계속 안 되면 운영자가 확인해 처리해요"를 띄우고 30초 간격으로 늦춥니다. 조회를 끄지는 않아서, 그 뒤에 등록이 끝나면 화면이 스스로 바뀝니다. 60초는 관리자 대조가 "등록 지연"으로 잡는 시간과 같아서, 사용자와 운영자가 같은 시점에 지연을 봅니다.
- ✦ **서버가 하지 않은 약속은 화면도 하지 않습니다.** 서버에는 "등록 실패" 상태가 없고, 끝내 안 되면 운영자가 처리합니다. 그래서 지연 안내에 "순서는 그대로"처럼 보장하는 말을 쓰지 않았습니다.
- ✦ **다른 탭에 가 있어도 상태를 따라갑니다.** 제한 시간은 탭을 떠나 있어도 흐르므로 조회도 함께 돌려서, 돌아왔을 때 이미 끝난 등록이 "처리 중"으로 남지 않게 했습니다.
- ✦ **결제 기한이 되는 순간 한 번 더 묻습니다.** 서버 시계보다 1초 늦게 물어, 만료 취소가 시작된 상태를 화면에 바로 반영합니다.

실패 유형별로 나눈 재시도와 백오프 정책

구현 조건

- 다시 보내도 결과가 같은 실패(로그인이 안 된 401, 잘못된 입력의 400 등)는 재시도하지 않아야 합니다.
- 서버가 힘든 순간에 모든 사용자가 곧바로 다시 보내 상황을 더 나쁘게 만들지 않아야 합니다.

구현

- **재시도 기준을 하나로 정했습니다.** 4xx는 재시도하지 않고, 5xx·타임아웃·네트워크 오류만 재시도합니다. 재시도 여부는 한 함수(`isRetryableError`)에서 판단합니다.
- **데이터 성격별로 캐시 정책 3가지를 만들었습니다.** 각 `useQuery` 에 값을 직접 쓰지 않고 이 정책 중 하나를 골라 펼쳐 씁니다.

정책	대상	staleTime	재시도
`catalog`	상품 목록·검색·상세	5분	2회
`account`	프로필·배송지	5분	1회
`live`	장바구니·알림 개수	0	1회

- **변경 요청(mutation)은 자동 재시도하지 않습니다.** 같은 요청이 두 번 반영될 수 있기 때문입니다.
- **이전 요청을 취소합니다.** 모든 조회 훅이 `queryFn` 의 `signal` 을 넘겨서, 검색 조건이 바뀌면 이전 요청이 취소됩니다. 늦게 도착한 이전 응답이 새 화면을 덮어쓰지 않습니다.

백오프

- **실패가 이어지면 점점 길게 쉬었다가 다시 묻습니다.** 실패한 요청을 바로 다시 보내면 이미 힘든 서버에 요청이 더 쏟아집니다. 그래서 평소 간격에서 시작해 실패할 때마다 2배로 늘립니다(예: 평소 3초 간격이면 3초 → 6초 → 12초 → 24초 → 30초). 30초에서 멈춥니다.

- **서버가 간격을 알려 주면 그보다 일찍 보내지 않습니다.** 응답 헤더의 `Retry-After`("몇 초 뒤에 다시 보내라")는 API 클라이언트가 읽어서 넘기고(실패는 에러의 `retryAfter`, 순번 조회는 `requestWithHeaders`), 쓰는 쪽이 그 값을 씁니다. 대기열 순번은 서버가 알려 준 `Retry-After` 마다 다시 묻고(그보다 자주 묻지 않습니다), 요청이 실패한 뒤에는 그 값을 대기 시간의 최솟값으로 삼습니다. 서버가 아무 말이 없을 때(타임아웃, 연결 끊김)는 위의 백오프로 스스로 간격을 늘립니다. 사용자들이 같은 순간에 한꺼번에 다시 묻지 않도록 대기 시간에 최대 1.5배까지 무작위로 더하는데, 위쪽으로만 흩어서 서버가 정한 값보다 짧아지는 일은 없습니다.
- **상한을 30초로 둔 이유가 있습니다.** 타임아웃 10초를 더해도 한 번에 약 55초라서, 대기열이 250초 안에 적어도 4번은 다시 묻습니다. 너무 길게 쉬면 내 차례를 놓칩니다.
- **폴링은 한 번의 조회 안에서 재시도합니다.** 주기마다 실패 횟수가 0으로 돌아가면 연속 실패를 셀 수 없어서, 실패하면 그 조회 안에서 계속 다시 묻고 간격을 늘립니다. 그동안 화면에는 에러 대신 "다시 확인하고 있어요" 안내만 보여줍니다.

모든 API 호출을 한 곳에서, 같은 모양으로: API 클라이언트

구현 조건

- 서버가 내려준 실패, 프록시의 에러 페이지, 타임아웃, 네트워크 끊김, 요청 취소를 화면이 같은 방식으로 다룰 수 있어야 합니다.
- 인증 헤더와 401 처리를 한 곳에서 해야 합니다.
- 사용자가 문의하면 서버 로그에서 어느 요청인지 찾을 수 있어야 합니다.

구현

서버에서 어떤 일이 생겨도 화면이 받는 것은 **데이터 아니면 에러 하나**입니다. 상황별로 이렇게 처리합니다.

response 값	client.ts 처리	화면에서 보임
`success: true` (정상 응답)	봉투를 벗겨 필요한 데이터만 꺼냅니다	데이터
`401` + `retryable: true` (일시 장애 표시)	재발급 없이 2초 뒤 같은 요청을 한 번 더 보냅니다	데이터 또는 에러
`401` (로그인이 풀림)	토큰을 다시 받고 같은 요청을 딱 한 번만 다시 보냅니다	성공하면 데이터, 또 401이면 에러

response 값	client.ts 처리	화면에서 보임
`success: false` + 에러 코드·메시지 (예: 잘못된 입력, 이미 접수됨)	실패 내용을 에러 하나로 바꿉니다	에러 (서버의 코드·메시지 그대로)
JSON이 아닌 본문 (프록시 에러 페이지 등)	"서버 응답을 처리할 수 없음"으로 바꿉니다	에러 (`INVALID_RESPONSE`)
응답 없음 (10초 초과, 연결 끊김)	"응답이 늦음" 또는 "네트워크 확인"으로 바꿉니다	에러 (`TIMEOUT` / `NETWORK_ERROR`)
요청 취소 (화면을 떠남)	실패가 아니라서 바꾸지 않고 그대로 둡니다	취소 (에러 화면이 뜨지 않음)

화면은 `ApiRequestError` 하나만 알면 됩니다. 실패 종류를 가려야 할 때만 에러 안의 `code` 를 보면 됩니다.

- **한 곳에서 처리합니다.** 서버가 실패도 JSON 봉투(`success / data / error / traceId`)로 주기 때문에 `validateStatus` 를 열어 두고 `unwrap` 한 곳에서 벗깁니다. 호출하는 쪽은 `instanceof ApiRequestError` 하나로 분기합니다. 재발급 요청 자체는 `skipAuthRefresh` 로 재발급을 다시 부르지 않게 했습니다.
- **문의 코드로 서버 로그를 찾게 했습니다.** 모든 요청에 `X-Request-Id` 를 실어 보내고 서버가 이를 `traceId` 로 이어 씁니다. 응답이 없어도 보낸 ID가 에러에 남아, 에러 화면에서 "문의 코드"로 보여줄 수 있습니다.
- **기본 규칙에 예외를 열어 두었습니다.** 타임아웃은 기본 10초이고, 사진 업로드와 결제 승인처럼 오래 걸리는 요청만 `timeout` 을 늘립니다. 실시간 알림(SSE)은 `axios` 가 스트림을 못 읽어 `fetch` 로 열되, 연결 전 실패는 같은 `ApiRequestError` 로 던집니다.

규칙을 문서가 아니라 lint로 강제

구현 조건

- FSD 레이어 규칙(`app → pages → widgets → features → entities → shared`)을 말로만 정하지 않고, 급할 때도 역방향 import가 생기지 않도록 도구로 지켜야 합니다.

구현

- `eslint-plugin-import-x` 의 `no-restricted-paths` 로 레이어별 import 금지 구역을 설정했습니다. 규칙을 어기면 커밋 단계(husky + lint-staged)에서 막힙니다.

- **서버 응답 형태(DTO)가 UI까지 새지 않게 했습니다.** DTO 타입은 `entities/*/api`와 `entities/*/model`에서만 import할 수 있도록 lint로 막았습니다. API 응답이 바뀌어도 고칠 곳은 `entities/*/api` 한 곳입니다.
- **경계 문제는 의존성 주입으로 풀었습니다.** `shared`의 API 클라이언트는 `entities/auth`를 import할 수 없습니다. 그래서 인증 헤더와 401 처리 함수를 앱 부팅 시점에 주입받는 방식(`configureApiAuth`)으로 해결했습니다.

트러블슈팅 🔧

응답을 못 받았을 때, 접수가 두 번 되지 않게

문제

네트워크가 끊겨 접수 응답을 못 받으면 접수가 됐는지 알 수 없습니다. 다시 누르면 예약이 두 번 생기거나, 이미 쓴 입장권으로 보내다 거절당할 수 있습니다.

해결

- 입장권을 SHA-256으로 바꾼 **멱등키**(`Idempotency-Key`)를 `sessionStorage` 에 두고, 같은 키로만 접수합니다. 같은 키로 다시 보내면 서버가 기존 예약을 돌려줍니다.
- 그래서 응답 자체를 못 받은 경우(`status 0`)에만 자동으로 **한 번** 다시 보냅니다. "변경 요청은 자동 재시도하지 않는다"는 원칙의 유일한 예외입니다.
- 그래도 못 받으면 진행 중 예약을 다시 조회해, 접수됐다면 버튼을 그 예약으로 바꾸고 "다시 눌러도 예약은 하나만 접수돼요"라고 알립니다. 입장권과 키는 지우지 않습니다.

색을 고를 때마다 슬라이더를 새로 만들던 상품 카드

문제

상품 카드에서 색을 고르면 그 색의 사진이 첫 장부터 보여야 합니다. 처음에는 첫 장으로 되돌리는 가장 간단한 방법으로 슬라이더에 `key` 를 사진 목록으로 걸어, 색을 누를 때마다 슬라이더를 지우고 처음부터 다시 만들었습니다. 이후 `Slider` 에 바깥에서 번호를 정하는 `activeIndex` 가 생긴 뒤에도 이 방식이 남아 있었고, 일부 클릭에서는 화면 반영이 한 프레임(60Hz 기준 약 17ms)가량 늦어졌습니다.

원인

`key` 가 바뀌면 React가 슬라이더를 새로 만들어, Swiper 생성·이벤트 연결·레이아웃 계산·사진 다시 그리기가 색을 누를 때마다 반복됩니다. 같은 주소의 사진은 브라우저 캐시를 쓰므로 낭비된 것은 내려받기가 아니라 슬라이더 생성이었습니다.

해결

`key`를 없애고, 카드가 현재 슬라이드 번호를 `activeIndex`로 들고 있다가 색을 고르면 `0`으로만 돌립니다. 이를 위해 공용 `Slider`가 `activeIndex`와 `onActiveIndexChange`를 받게 했습니다. 색을 고르면 첫 장이 보이는 동작은 그대로입니다.

측정

- 수정 전 커밋과 지금 버전을 같은 조건(프로덕션 빌드, 목 서버, 카드 1개)에서 색 버튼을 80번 눌러 비교했습니다. 갱신까지의 시간은 `requestAnimationFrame` 두 번으로, 슬라이더 재생성은 `MutationObserver`로 셌습니다.
- **슬라이더 재생성은 80번 중 80번에서 0번**이 됐습니다. 갱신까지 가장 느린 5%(p95)는 39~44ms에서 30ms로, 최댓값은 48~50ms에서 31ms로 내려갔습니다. 중앙값(약 29ms)은 같아서, 평균 반응 속도는 그대로이고 가끔 늦던 경우가 사라진 것입니다. 카드가 많은 목록이나 저사양 기기는 측정하지 않았습니다.

이미지 로딩도 함께

목록·랭킹 카드 사진에 `loading="lazy"`를 달고 바뀐 목록만 미리 받게 해, 홈 첫 방문 사진 요청을 **186개에서 19~93개**로 줄였습니다. 홈 배너(LCP)는 `fetchPriority="high"`로 먼저 받습니다.

로그아웃해도 이전 사용자의 대기열 정보가 남는 문제

문제

한 브라우저에서 A가 대기열에 선 채로 로그아웃하고 B가 로그인하면, B의 화면에 A의 대기열 pill이 그대로 떠 있었습니다. B가 그 입장권으로 접수하려 해도 "입장권이 유효하지 않다"는 안내만 받았습니다.

원인

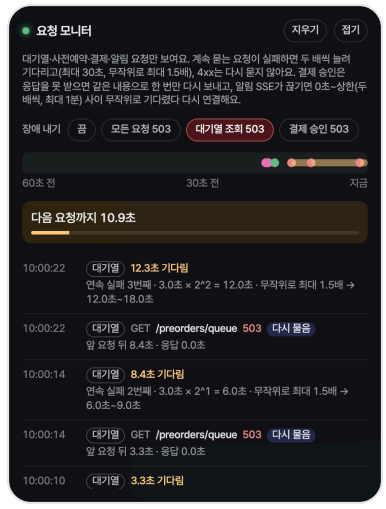
로그아웃할 때 서버에서 받아 둔 데이터(쿼리 캐시)만 지우고, 대기열 정보(메모리와 `sessionStorage`에 저장)는 지우지 않았습니다. 그래서 로그아웃한 뒤에도 줄 상태를 확인하는 요청이 계속 나갔습니다.

해결

- 로그아웃되는 순간(`onSessionCleared`) 쿼리 캐시와 대기열 정보(순번, 입장권)를 한곳에서 함께 지웁니다. 줄에 서 있었다면 "로그아웃되어 대기열을 닫았어요"라고 알립니다.
- 쿼리 캐시 키에는 사용자 구분이 없어서, 로그아웃 없이 같은 브라우저에서 다른 계정으로 로그인할 때도 이전 사용자의 캐시를 지웁니다.

검증과 협업 ✅

- **커밋 규칙**: commitlint + husky로 커밋 메시지 형식 강제. 커밋 본문에 변경 이유를 남김
- **Storybook · Vitest**: 컴포넌트 상태별 화면(로딩·빈·에러·장애)을 스토리의 `play` 함수로 실제 브라우저(Chromium)에서 확인하고(스토리 190개), 돈·재시도가 걸린 순수 로직(결제 승인, 대기열 백오프, 페이지 이어 받기 등)은 단위 테스트로 검증합니다(37개 파일). 모두 팀 전체 기준입니다.
- **요청 모니터 (직접 만든 개발 도구)**: 백오프와 `Retry-After`가 의도대로 동작하는지 개발자 도구 없이 화면 옆에서 보려고 만들었습니다. 대기열·사전예약·결제·알림 요청과 실패 뒤 기다린 시간을 60초 타임라인에 쌓고, 기다린 시간이 어디서 나왔는지 풀어서 보여줍니다. 목 서버에서는 [장애 내기]로 모든 요청·대기열 조회·결제 승인을 503으로 실패시킬 수 있습니다. API 클라이언트는 요청 흐름에 끼어들지 않고 기록만 남깁니다.



대기열 조회를 503으로 실패시킨 모습. 연속 실패가 이어질수록 기다리는 시간이 늘어납니다(3.3초 → 8.4초 → 12.3초). 기본 3초에 2배씩 늘리고, 무작위로 최대 1.5배를 더합니다.

- **Playwright E2E**: 대기열·접수·결제·여러 창·서버 오류를 브라우저로 끝까지 따라가는 시나리오 65개(12개 파일, 운영자 시나리오 포함). 목 서버로 돌려서 `.env.local`과 무관하고, 토스 결제창은 가짜로 바꿔 실행. 확인한 내용은 아래 표에 정리했습니다.

Playwright E2E로 확인한 시나리오 (고객 화면 11개 파일, 60개)

파일	테스트	확인하는 것

파일	테스트	확인하는 것
`preorder-payment`	3	결제가 실패해도 예약 순번 유지, 결제 뒤 뒤로 가기, 구매 확정 기한이 지나면 취소 처리
`preorder-windows`	3	탭을 복제해도 같은 예약으로 이어짐, 다른 창에서 로그아웃하면 대기열이 닫힘, 로그아웃하면 줄·입장권 저장소를 비움
`server-error`	3	서버가 실패해도 홈 화면은 남고, 상품 상세는 결제·장바구니 버튼을 그리지 않음
`login` , `review-write`	2 + 2	새로고침해도 로그인 유지, 배송 완료 주문에만 리뷰 쓰기 표시
`preorder-open` , `preorder-mobile`	1 + 1	오픈 시각에 새로고침 없이 버튼이 열림, 휴대폰 화면에서 하단 탭부터 결제 페이지까지

운영자 화면 시나리오 5개(`admin.spec.ts`)는 팀원과 함께 작성했고 위 표에 넣지 않았습니다.

- **CodeRabbit AI 코드 리뷰**: PR마다 리뷰를 받고, 지적 사항을 반영한 커밋 12건 (접근성, 위젯 경계, 세션 보안 등)
- **AI 활용 · 규칙으로 남기기**: Claude Code가 프로젝트 규칙을 어긴 코드를 제안하면 고치는 데서 끝내지 않고, `CLAUDE.md` 와 경로별 `.claude/rules/` (styling · data-layer · routing · jsx)에 "겪은 사례"로 남겨 같은 코드가 다시 나오지 않게 함
- **역할별 서브에이전트 7종**: 읽기 전용 리뷰어(`fsd-reviewer` , `auth-payment-guard`), 검증(`test-runner` , `ui-verifier`), 작업 보조(`api-spec-aligner` , `review-responder` , `pr-writer`). 어떤 에이전트도 커밋·푸시·PR 답글은 하지 않고 제안까지만 함
- **브라우저 검증 (Chrome MCP)**: AI의 "됐다"는 답변만 믿지 않고 실제 브라우저로 화면, 콘솔 에러, 네트워크 요청을 확인하고 모바일·데스크톱 폭을 바꿔 반응형까지 점검
- **OpenWiki · Figma MCP**: 기능별 흐름과 코드 위치를 정리한 위키를 자동 생성하고, Figma 디자인과 구현 화면을 서로 대조

E

Ephyra

2026.08.14 ~ 2026.09.04 (3주)

Site ↗ GitHub ↗

Ephyra

개요 📖

에피라 · AI 통신 요금제 상담 · 가입 플랫폼 · LG유플러스 유레카 프론트엔드 개발자 과정 종합프로젝트. 데이터·통화·예산·OTT 이용 성향을 채팅으로 입력하면 AI가 요금제를 추천하고, 비교·가입·리워드까지 하나의 채팅 흐름으로 연결하는 서비스

- ✦ **역할** 조장 · PL · Frontend · Design
- ✦ **인원** FE 4인 (전원 FE, 일부 BE·LLM 겸임)
- ✦ **기여도** 커밋 164개 (팀 전체 496개 중 2위) · 현재 코드의 약 28% · PR 13건

역할 👨‍💻

- ✦ **기능 상담 레포트**: 생성 데이터 구조 설계, 레포트 카드·상세 화면(추천·비교·변경 카드, 요약 카드)
- ✦ **기능 회원가입**: 채팅 흐름 안에서 진행되는 회원가입, 로그인 모달, 비로그인 사용자 로그인 유도 정책
- ✦ **기능 요금제 추천 입력 UI**: 드롭다운·숫자 입력을 칩 선택 방식으로 개편
- ✦ **기능 리워드·게임**: 출석 룰렛, 반응속도 게임, 출석·배지·상품 교환 API 연동
- ✦ **기능 마이페이지 사용량**: 최근 3개월 사용량 차트(Chart.js 기반 `LineChart`), 데이터·통화 사용량 섹션
- ✦ **공통·품질 공통 UI**: Button, Input, Card, Modal, Tab, LineChart와 Storybook 스토리

- **공통·품질 QA 리포트와 회귀 테스트**: 서비스 전체 QA를 진행하고 Playwright 회귀 테스트 3종 작성
- **공통·품질 팀 운영**: 조장·PL로 일정과 역할 조율, CodeRabbit AI 코드 리뷰 도입

Skills 🛠️

- React 19, TypeScript, Vite, Tailwind CSS 4, TanStack Query, Zustand
- Supabase (Auth · PostgreSQL · Edge Functions), OpenAI API, Storybook, Playwright, MSW, Vercel

핵심 구현 💡

LLM에게 맡길 것과 맡기지 않을 것: 상담 레포트 구조 개편

문제

처음 상담 레포트는 대화 전체를 LLM에 넘기고, 요금제명·추천 사유·절감액까지 LLM이 대화 텍스트에서 다시 추론하게 하는 구조였습니다.

- 클라이언트가 이미 정확히 알고 있는 구조화된 값(추천 요금제, 월 절감액)을 LLM이 다시 추측하다 보니 틀릴 여지가 있었습니다.
- 상담 중 요금제를 비교하거나 실제로 가입·변경한 기록은 레포트에 아예 반영되지 않았습니다.

판단

정확해야 하는 값은 코드가, 요약은 LLM이 맡도록 역할을 나눴습니다.

섹션	데이터 출처
추천 (`recommendedPlans`)	추천 요청마다의 라운드 배열. 채팅 메시지에 이미 있는 구조화 데이터
비교 (`comparedPlan`)	마지막 비교 결과
변경 (`changedPlan`)	실제 가입·변경된 요금제 + LLM이 생성한 "기존 대비 좋은 점"
기타 (`otherNotes`)	LLM이 생성한 자유 서술 요약

구현

- 같은 조건 입력에서 나온 추천들을 한 라운드로 묶기 위해, 정보 입력 폼을 새로 제출할 때마다 `groupId` 를 발급했습니다. 이후 퀵 리플라이로 다시 질문하면 직전 라운드의 `groupId` 를 이어받습니다.
- LLM에 넘기던 Q&A에서 이미 구조화된 추천 조건(나이·데이터·예산·OTT)이 중복되던 부분을 제거했습니다.
- 개편 전에 저장된 레포트는 기존 관계형 조인 결과로 자동 폴백되도록 해서, 데이터 마이그레이션 없이도 이전 레포트가 깨지지 않게 했습니다.

QA 리포트와 Playwright 회귀 테스트

배경

마감 3일 전, 팀원 4명이 나눠 만든 기능이 하나의 채팅 화면에 합쳐지면서 기능 사이의 문제가 보이지 않는 상태였습니다.

진행

- 목업이 아니라 **실제 Supabase와 실제 OpenAI Edge Function**을 대상으로, Chrome MCP로 AI가 실제 브라우저를 조작하게 해서 기능별 정상 흐름을 처음부터 끝까지 실행했습니다.
- 발견한 문제를 P0~P3로 분류해 QA 리포트로 정리했습니다(P0 2건, P1 2건, P2 3건, P3 2건).
  - **P0**: "기타 상담" 모드가 어떤 질문에도 같은 고정 문구만 반환함 (LLM 미호출 + 키워드 라우터가 이전 모드 유지)
  - **P0**: Error Boundary가 하나도 없어서, 지연 로드 청크 하나가 실패하면 화면 전체가 백지가 됨 (실제 재현)
  - **P1**: README에 적힌 접근성 기능이 코드에 없음
- 재현한 시나리오를 Playwright 회귀 테스트 3종으로 남겼습니다. 테스트는 QA 전용 dev 서버를 스스로 띄운 뒤 헤드리스 Chromium에서 실행되어, 사람이나 AI가 매번 다시 확인하지 않아도 됩니다.
  - 게스트 스모크 테스트
  - 회원가입 → AI 추천 → 요금제 가입 골든 패스
  - "기타 상담" 버그 재현: `test.fail`로 기록해 두고, 버그가 고쳐지면 통과로 전환되도록 함

비로그인 사용자 정책: 어디까지 열어 둘 것인가

판단

AI 상담은 로그인 없이 체험할 수 있어야 첫 진입 장벽이 낮습니다. 하지만 OpenAI 호출에는 비용이 듭니다.

구현

- 비로그인 상태로 AI 응답이 5회 쌓이면 로그인 모달을 한 번 자동으로 띄웁니다.
- 그 이후에는 퀵 리플라이 등 어떤 버튼을 눌러도 실제 동작 대신 로그인 모달이 뜹니다.

- ✦ 비회원이 레포트를 생성할 수 없도록, 일반 대화 경로에 빠져 있던 로그인 체크를 추가했습니다. 요금제 추천 경로에는 이미 체크가 있었습니다.
- ✦ 채팅 밖(헤더)에서 회원가입을 누르면 채팅 홈으로 이동한 뒤 가입 흐름을 이어서 시작하도록 했습니다. 페이지 이동 대신 `useSignupIntentStore` 신호를 사용합니다.

추천 조건 입력을 칩 선택으로

문제

나이·우선순위는 드롭다운, 데이터·예산은 숫자 입력이었습니다. 사용자가 자기 데이터 사용량을 숫자로 정확히 모르는 경우가 많았습니다.

구현

- ✦ 모든 입력을 칩 버튼으로 통일하고, "무관"과 "미확인" 선택지를 추가했습니다.
- ✦ "무관"을 고르면 실제 제출값에서는 필터 없음과 같게 처리하되, 채팅 요약에는 사용자가 고른 그대로 보이도록 했습니다.
- ✦ 서버(Edge Function)가 "값 없음"과 "명시적으로 무관을 선택함"을 구분할 수 있도록 `skippedFields`를 추가했습니다. 그래서 사용자가 이미 "무관"이라고 답한 항목을 AI가 다시 묻지 않습니다.

트러블슈팅 🔧

요금제를 바꿨는데 채팅에는 예전 요금제가 남는 문제

원인

같은 "현재 요금제" 데이터를 두 곳에서 읽고 있었습니다. 요금제 가입·변경 시 TanStack Query 캐시(`['plans','current']`)만 갱신했는데, 채팅은 별도의 Zustand 스토어 (`useSubscriptionStore`)를 참조하고 있었습니다. 그래서 레포트와 비교 화면에 예전 요금제가 계속 "현재 요금제"로 표시됐습니다. QA에서 데이터 출처가 두 곳으로 나뉜 구조를 먼저 발견해 두었습니다.

해결

신청이 성공하면 Zustand 스토어도 함께 갱신하도록 수정했습니다.

남은 과제

근본적으로는 서버 데이터를 TanStack Query 한 곳에서만 읽어야 합니다. 마감 일정 때문에 두 곳을 동기화하는 선에서 멈췄습니다.

스크래치 게임 보상이 여러 번 지급되는 문제

원인

클리어 판정 후에도 `pointermove` · `pointerup` 이벤트 리스너가 남아 있어서, 판정 함수가 계속 실행되며 `onWin` (배지 지급)이 중복 호출됐습니다.

해결

`hasCleared` 플래그로 클리어 처리를 1회로 제한하고, 클리어 시 `pointermove` 리스너를 제거했습니다.

Tailwind 색상 클래스가 적용되지 않는 문제

원인

마이페이지 사용량 바에서 항목별 색을 `text-${chooseColor(label)}` 처럼 문자열을 조합해 만들었습니다. Tailwind는 빌드할 때 소스 코드에서 완성된 클래스명을 정적으로 찾아 CSS를 생성하기 때문에, 조합된 클래스는 CSS가 만들어지지 않아 색이 보이지 않았습니다.

해결

항목별로 완성된 클래스명(`text-reward-active` , `bg-reward-active` 등)을 `COLOR_CLASSES` 매핑 객체에 그대로 적어 두고, 라벨을 키로 선택하는 방식으로 바꿨습니다.

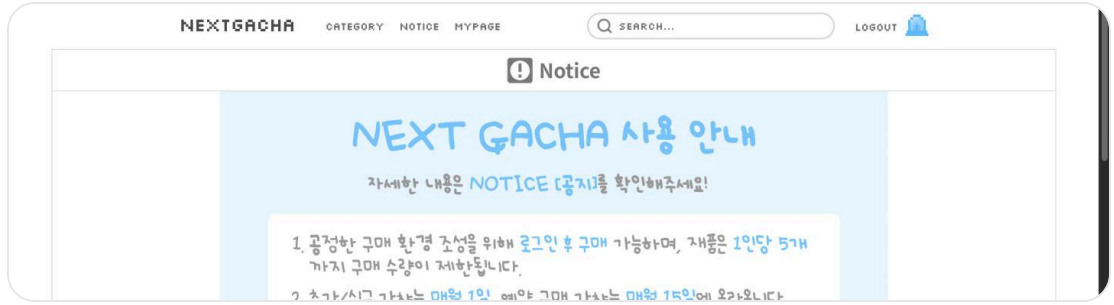
검증과 협업 ✅

- **Storybook**: 공통 UI 컴포넌트 카탈로그 (팀 전체 31개 스토리)
- **CodeRabbit AI 코드 리뷰**: 직접 도입하고 모든 브랜치를 검사하도록 설정
- **Playwright**: 실제 백엔드 대상 회귀 테스트 3종
- **AI 활용**: Chrome MCP로 AI가 실제 브라우저를 조작하게 해서 기능별 정상 흐름을 처음부터 끝까지 실행하며 QA를 진행하고, 찾은 문제의 재현 시나리오는 Playwright 테스트 코드로 고정

NEXT GACHA

2025.09 ~ 2026.02 (약 6개월) + 2026.09 구조 리팩토링 · 2026.10 성능 개선

Site ↗ GitHub ↗



개요 📖

1인 5개 구매 제한 가챠 쇼핑몰 · 개인 프로젝트 · 1인 개발. 되팔이의 대량 구매를 막기 위해 같은 상품을 1인당 최대 5개까지만 살 수 있게 만든 온라인 가챠 쇼핑몰

- **역할** 기획 · Frontend · Backend · Design
- **인원** 1인 개발
- **규모** 커밋 129개, API Route 20여 개, 사용자 화면과 관리자 백오피스 포함

역할 👨‍💻

- **Frontend** 페이지별 렌더링 방식(Server/Client) 결정, 데이터 조회·캐싱 구조, 장바구니·결제·마이페이지
- **Backend** Next.js API Route로 BFF 구성, MongoDB Atlas CRUD, 카카오 로그인 (NextAuth), 토스페이먼츠 결제 승인
- **관리자** 상품·재고 관리, 주문 상태 일괄 변경, 이벤트 할인, 통계, 오류 수집 페이지
- **Design** UI/UX 디자인 (Figma)

Skills 🛠️

- Next.js 16 (App Router), TypeScript, TanStack Query, Jotai, Vanilla Extract, NextAuth (카카오 OAuth)

- MongoDB Atlas, TossPayments, Vercel Blob, MUI X Charts, Jest + React Testing Library, Vercel

기획 배경 📋

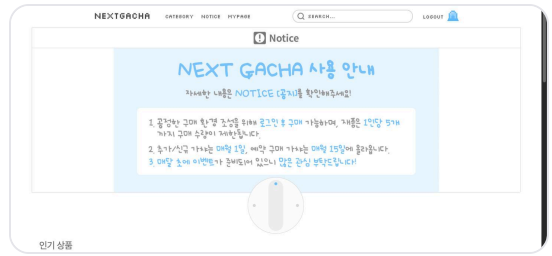
정가 상품을 대량으로 사서 웃돈을 붙여 되파는 사람들 때문에, 정작 그 상품을 원하는 사람이 더 비싸게 사야 하는 문제가 있습니다.

그래서 **같은 상품을 1인당 최대 5개**까지만 살 수 있게 했습니다. 기준은 전시용, 보관용, 예비용 2개, 선물용입니다. 배송이 끝나기 전까지는 주문한 수량도 이 한도에 포함되고, 배송이 완료되면 다시 5개까지 살 수 있습니다.

시스템 구성도 🏗️

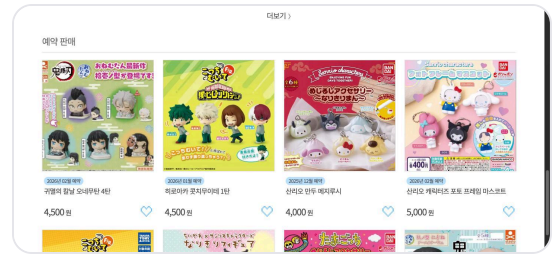
NEXT GACHA 시스템 구성도
Next.js 16 (App Router) · 같은 상품 10만장 최대 5개 제한 쇼핑몰
브라우저
고객
관리자
ADMIN_EMAILS
Next.js 클라이언트
FSD: views → widgets → features → entities → shared
• TanStack Query: 서버 상태 캐시
• Jotai: 상세 페이지의 선택 수량(메모리)
• Vanilla Extract: 컴파일 타임 스타일
• API는 /api/... 상대 경로로 호출
결제 흐름
1. /checkout/[orderId]
2. 토스 SDK 결제창 (브라우저 ↔ Toss)
3. 성공 시 /success (paymentKey)
4. POST /api/postPaymentsConfirm
5. 서버 승인 DB 반영 (이때 서버 공용 로직)
6. 실패 이탈 시 /fail
금액·포인트는 클라이언트 값을 믿지 않고 서버가 DB로 다시 계산
5개 제한은 세 번 검사
① 클라이언트 훅(즉시 안내)
② 장바구니 API (checkCartLimit)
③ 결제 승인 (verifyOrder → reducePoint)
모든 요청
Vercel · Next.js 서버
proxy.ts (Next.js middleware)
• /api/protected/* → 로그인 아니면 401
• 관리자 API 7종 → ADMIN_EMAILS가 아니면 403
• /manager/* → 비관리자는 / 로 이동 (/manager, /manager/statistics는 공개) · /mypage/* → /signin
• 페이지 요청마다 POST /api/track (event.waitUntil(prefetch 제외)
통과
통과
페이지 (Server Components)
• /main · /search · /notice
• /[code] (info · review · qna 탭)
• /mypage (like · cart · history)
• /(payment) checkout · success · fail
• /manager (product_add · product_manage · order · user · event · error · statistics)
자기 /api를 fetch하지 않음 (세션 쿠키가 실리지 않아 401/403). 공용 서버 함수로 DB를 직접 조회
API Routes (BFF) · 20여 개
• /api/protected/* (로그인 필요): getHistory · getLike · updateLike · postCart · postReview · postQna · deleteQuestion …
• 관리자 7종: postProduct · postEditProduct · postEvent · postCheckToSending · postSendingToFinish · getOrderSort · getError
• 결제: postPaymentsConfirm
• 그 외: getProduct · getOneProduct · getReview · getQna · getCart · updateCart · getUser · updateAddress · getStatistics · postCheckStock · postError · track
직접 호출
호출
NextAuth v5 (Kakao만)
첫 로그인 시 user 문서 생성 (포인트 0)
user 생성
서버 공용 로직 · shared/api/mongodb.ts
• 단일 MongoClient (모듈 로드 시 연결) · 컴렉션 핸들을 하나씩 export
• features/purchase/lib: validateStock → verifyOrder → (Toss 승인) → withTransaction(reducePoint → reduceStock → addOrder → resetCart) · 가격·배송·포인트 규칙은 orderPolicy.ts
• features/order-manage/lib: findOrdersByStatus (페이지와 API가 함께 사용) · 상태 변경은 status 필터로 중복 클릭을 무효화
트랜잭션 실패 시 Toss 결제를 취소하고, 취소마저 실패하면 errorCollll에 기록
외부 서비스 · 데이터
승인 · 취소
TossPayments
• 결제창 (브라우저 SDK)
• 서버: 승인 confirm · 취소 cancel
• 결제 실패·환불 시 취소
put
Vercel Blob
상품 이미지 images/{번호}.png (postProduct의 put)
OAuth
Kakao OAuth
이메일을 주지 않으면 로그인 거부
MongoDB 드라이버
MongoDB Atlas (gachaDB)
user · product · cart · order
review · qna · error · counter · pageViews
Atlas Trigger
매월 이벤트 상태 초기화

주요 화면 🖼️



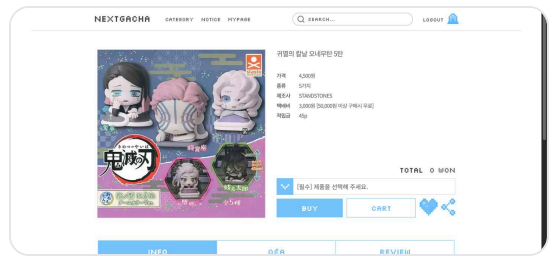
홈

사용 안내 배너 아래에 인기·신규·예약 상품을 보여줍니다. 서버 컴포넌트로 세 섹션을 동시에 조회합니다.



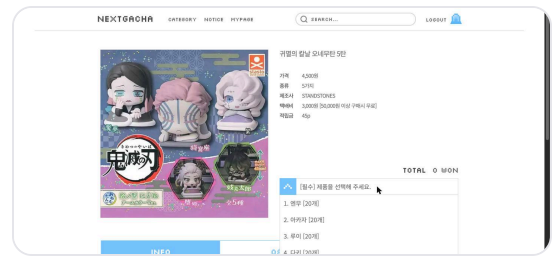
예약 판매

홈 아래쪽의 예약 판매 목록입니다. 상품마다 예약 월을 태그로 보여줍니다.



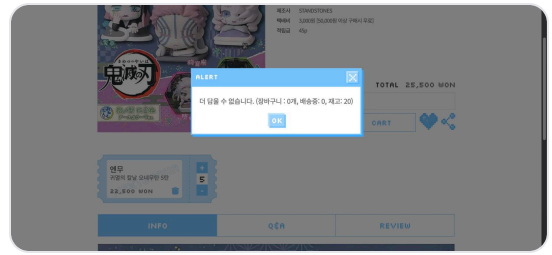
상품 상세

가격·택배비·적립금과 옵션 선택, 구매·장바구니·찜. 아래 탭에서 정보·Q&A·리뷰를 봅니다.



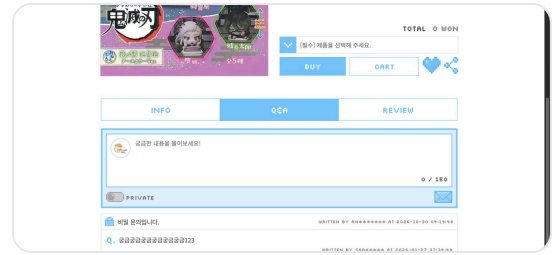
옵션 선택

옵션별 남은 재고를 함께 보여줍니다. 고른 옵션은 수량 카드로 담깁니다.



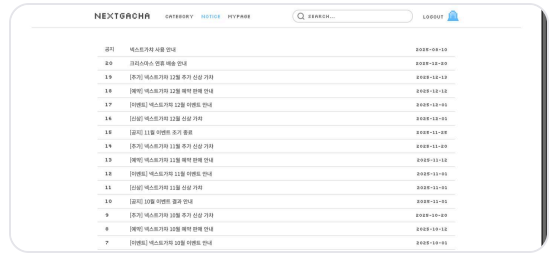
5개 제한 안내

수량을 5개 넘게 올리면 안내가 뜹니다. 장바구니·배송 중 수량과 재고를 함께 계산해, 같은 상품을 1인당 5개까지만 담게 합니다.



상품 Q&A

비밀 문의는 내용을 가리고, 작성자 이메일은 일부만 보이게 가립니다.



공지

월별 신상·예약 판매·이벤트 공지 목록입니다.



관리자 · 배송 상태 변경

관리자가 주문을 확인한 뒤 택배사와 송장번호를 넣어 배송중으로 바꾸거나, 사유를 적어 배송을 거절합니다. 여러 건을 선택해 한 번에 바꿀 수도 있습니다.

핵심 구현 💡

장바구니 수량 변경: 서버 캐시 재검증에서 클라이언트 캐시로

처음 설계

상태 관리 라이브러리 없이 App Router 캐싱만 쓰려고, 수량을 바꿀 때마다 서버 API를 호출하고 `revalidateTag`로 캐시를 무효화한 뒤 다시 조회했습니다.

문제

- 수량 버튼을 누를 때마다 캐시 무효화 → 재조회가 반복되어, 서버 응답이 올 때까지 화면 숫자가 바뀌지 않았습니다.
- 연속으로 클릭하면 각 요청이 이전 응답(stale state) 기준으로 나가서, 5개 한도를 넘는 요청이 생길 수 있었습니다.

판단

장바구니처럼 자주 누르는 기능은 즉각적인 반응이 더 중요하다고 보고, 번들 크기가 늘어나는 것을 감수하고 TanStack Query로 바꿨습니다.

구현 · features/cart/model/useCart.ts

- **요청 전 검증**: 증가 버튼을 누르면 서버로 보내기 전에 `장바구니 수량 + 배송 중 수량 < min(5, 재고 + 배송 중 수량)`을 확인합니다. 한도를 넘으면 요청을 보내지 않고 이유(재고, 배송 중 수량, 최대 수량)를 안내합니다.
- **연속 클릭 방지**: 요청이 처리되는 동안(`isPending`)에는 추가 클릭을 무시합니다.
- **낙관적 업데이트**: `onMutate`에서 화면 숫자를 먼저 바꾸고, 실패하면 `onError`에서 이전 값으로 되돌립니다. `onSettled`에서 서버와 다시 맞춥니다.
- **삭제 확인**: 수량이 1일 때 감소 버튼을 누르면 삭제 여부를 먼저 묻습니다.

FSD 구조로 리팩토링

6개월 동안 기능을 붙이면서 `components/atoms·molecules·templates`와 `app/hooks`에 코드가 섞여 있었습니다. 기능 하나를 고치려면 여러 폴더를 오가야 했습니다.

- **FSD 레이어로 이전**: `shared → entities → features → widgets → views` 순서로 5단계에 걸쳐 옮기고, 레이어 규칙을 어긴 import(BuyBtn, HeartBtn, like)를 정리했습니다.
- **스타일 전환**: 파일 곳곳의 inline style을 Vanilla Extract `css.ts`로 8단계에 걸쳐 모두 옮겼습니다.
- **단계별 커밋**: 각 단계를 별도 커밋으로 나눠서, 문제가 생기면 그 단계만 되돌릴 수 있게 했습니다.

proxy 기반 접근 제어

- ** `/api/protected/*` **: 로그인하지 않으면 401
- **관리자 API** (상품 등록·수정, 주문 상태 변경, 오류 조회 등): `ADMIN_EMAILS`에 등록된 계정이 아니면 403
- **관리자 페이지**: 메인 메뉴와 통계는 포트폴리오 시연용으로 공개하고, 나머지는 관리자만 접근하도록 리다이렉트
- **Q&A**: 수정·삭제 시 작성자 본인인지 소유권을 검증

운영을 고려한 관리자 백오피스

- **주문 관리**: 주문 확인 → 배송 중 → 배송 완료 상태를 여러 건 한 번에 변경. 배송 완료 시 해당 상품을 다시 구매할 수 있도록 배송 중 수량(`keep`)을 줄임
- **통계**: 검색어, 카테고리, 상품 단위의 방문 기록을 proxy에서 수집해 MUI X Charts로 시각화
- **오류 수집**: 사용자가 보낸 오류 내용을 발생 URL과 함께 확인
- **이벤트**: 카테고리별 상품 할인율 적용·수정. 매월 이벤트 상태는 MongoDB Atlas Trigger로 초기화

홈 서버 렌더링 전환으로 Lighthouse 성능 개선

측정

Lighthouse 13.5.0(headless Chrome) 모바일·데스크톱 프리셋, 각 3회 측정의 중앙값. 수정 전후를 같은 설정으로 비교하고, 운영 배포 후 다시 쟀습니다.

병목 진단

운영 홈의 모바일 LCP 4.1s 중 3.17s는 이미지를 받는 시간이 아니라 **요청하기 전 대기 시간** (resource load delay)이었습니다. 다운로드 자체는 16ms였습니다.

- 홈이 `'use client'` 컴포넌트라, JS 로드·하이드레이션 후 `useEffect`에서 `/api/getProduct`를 인기 → 신규 → 예약 순서로 **직렬 3회** 호출했습니다(응답당 약 700ms). 상품 이미지는 그 뒤에야 요청됐습니다.
- 상품 카드가 늦게 들어오면서 "더보기" 버튼이 밀려 CLS 0.13이 생겼습니다.

수정

- **홈을 서버 컴포넌트로 전환**: `HomePage`를 `async` 서버 컴포넌트로 바꾸고 세 섹션을 `Promise.all`로 동시에 조회했습니다. 서버 컴포넌트가 자기 `/api`를 fetch하면 세션 쿠키가 실리지 않아서, 조회 로직을 공용 서버 함수(`findProducts`)로 빼 API 라우트와 함께 쓰도록 했습니다.
- **배너 첫 장 우선 로딩**: 첫 슬라이드만 `priority`로 바꿔 HTML에 preload가 들어가게 하고, 나머지는 lazy로 뒀습니다.
- **폰트를 variable font 한 벌로**: Noto Sans KR을 400/500/700 세 벌 대신 굵기 조절형 한 벌로 바꿔 `@font-face` 376개 → 128개, 해당 CSS 73KB → 24KB로 줄였습니다.
- 측정 중 `pnpm start` 스크립트에 존재하지 않는 옵션(`--webpack`)이 있어 로컬 서버가 뜨지 않던 버그도 찾아 고쳤습니다.

결과 (운영 기준)

지표	모바일 전 → 후	데스크톱 전 → 후
성능 점수	73 → 80 (회차별 78~91)	90 → 99
LCP (실측)	3.28s → 1.78s	0.89s → 0.98s
CLS	0.13 → 0.001	0.152 → 0.007

측정으로 되돌린 것

Lighthouse 제안을 따랐다가 오히려 나빠진 두 가지를 되돌렸습니다.

- `experimental.inlineCss` : 폰트 CSS 231KB가 HTML에 들어가 HTML이 583KB가 되고, 모바일 LCP가 8s대로 나빠졌습니다.
- AVIF 이미지 포맷: 로컬 이미지 최적화기에서 인코딩이 끝나지 않아 Lighthouse가 타임아웃을 냈습니다.

개선책을 하나씩 넣고 매번 재야, 어느 수정이 효과를 냈는지 가릴 수 있다는 것을 확인했습니다.

배포 후 문제

첫 배포 직후 hydration 에러(React #418)가 났습니다. 예약 태그의 월을 서버(UTC)는 "2025년 12월", 브라우저(KST)는 "2026년 01월"로 다르게 그렸기 때문입니다. `toKoreaTime`으로 양쪽을 KST 기준으로 맞춰 해결했습니다.

남은 한계

- 페이지 폭이 1312px로 고정돼 있어 모바일에서 상품 이미지와 배너가 모두 첫 화면으로 계산됩니다. 모바일 점수를 올리는 가장 큰 레버는 반응형 레이아웃입니다.
- 데스크톱 실측 LCP는 그대로입니다. 서버가 DB 조회를 마친 뒤 HTML을 보내므로, 늘어난 TTFB가 줄어든 대기 시간을 상쇄했습니다.

검증✅

- **Jest + React Testing Library**: 공통 버튼(클릭, disabled, 링크 렌더링)과 금액 포맷 유틸 테스트

Experience 💼

- **LG U+ 유레카 프론트엔드 개발자 과정** (멀티캠퍼스) · 2026.04.06 - 2026.10.28 — Ephyra, Nova
- **주식회사 디엔소프트** 프론트엔드 개발자(계약직) · 2024.01 - 2024.04 — Vue.js 관리자 페이지, 상품 웹사이트 리뉴얼
- **디엔소프트** 학교 현장실습 · 2023.09 - 2023.12 — Vue.js 박람회 이벤트 사이트

Activities

- **2024년 국방 공공데이터 활용 경진대회** · 2024.04 - 2024.06 — 4인 팀 프론트엔드
- **코딩 동아리 '씨부엉'** · 2020.03 - 2024.02 — 스터디 운영, 프로젝트 PM
- **교내 부트캠프** · 2022.06 - 2022.07 — 프론트엔드 단독 담당

Education

- **한국공학대학교 소프트웨어공학과** · 2019.03 - 2024.02 졸업 · 학점 3.0 / 4.5

Certificated

- 정보처리기사 - 한국산업인력공단 (2024.12)

Awards

- 교내 부트캠프 2위
- 졸업작품 A-Diary 우수팀 선정 (관람객 투표·교수진 심사)

Thank You